# Instituto Politécnico Nacional

Escuela Superior de Cómputo

Ingeniería en Sistemas Computacionales

IoT – Embedded Systems

## “Makefiles”


**Alumno:**

León Flores Daniela Alejandra

**Boleta:**

2023630293

**Profesor:**

Miguel Ángel Aleman Arce

**Grupo:** 6CM3

# Objetivo

Aplicar el uso de un Makefile en un proyecto propio, automatizando su compilación, ejecución, generación de reportes y limpieza de archivos, además de documentar el proyecto mediante un repositorio personal con README.md

# Desarrollo

En esta actividad se uso el repositorio del profesor como ejemplo, la idea fue aplicar la misma lógica vista en ese ejemplo: usar makefile para automatizar tareas comunes dentro de un proyecto. La idea principal fue crear un flujo automatizado que permita compilar un programa en c, ejecutarlo con un archivo de datos y generar un reporte con los resultados de dicho análisis.

El programa recibe como entrada un archivo CSV con datos de temperatura, humedad y CO2. Luego se procesan las muestras, calcula estadísticas y genera u reporte en formato txt.

El archivo de entrada utilizado fue: *data/sensor_data.csv*

El reporte generado por el programa fue: *reports/iot_report.txt*

EL programa calcula los siguientes datos: número de muestras procesadas, temperatura promedio, humedad promedio, promedio de CO2, temperatura máxima registrada, alertas por temperatura mayor a 30 °C, alertas por CO2 mayor a 900 ppm.

## Estructura del repositorio

```
Makefiles/
├── Makefile
├── README.md
├── .gitignore
├── data/
│   └── sensor_data.csv
├── include/
│   └── sensor.h
├── reports/
│   └── .gitkeep
└── src/
    ├── main.c
    └── sensor.c
```

### Makefile

El archivo Makefile es el elemento principal de automatización del proyecto. En él se definen los comandos necesarios para compilar, ejecutar, generar reportes, mostrar resultados y limpiar archivos generados.

Gracias a este archivo, no es necesario escribir manualmente todos los comandos de compilación cada vez que se modifica el programa.

### src/main.c

El archivo main.c funciona como punto de entrada del programa. Su función principal es recibir los argumentos de entrada, llamar a la función encargada del procesamiento de datos y mostrar en consola un resumen del análisis realizado.

Este archivo permite conectar la ejecución del programa con el archivo CSV de entrada y el archivo de reporte de salida.

### src/sensor.c

El archivo sensor.c contiene la lógica principal del proyecto. En este archivo se abre el archivo CSV, se procesan las lecturas, se calculan los promedios, se detectan alertas y se genera el reporte final. Es decir, este archivo transforma los datos de entrada en información útil para el usuario.

### include/sensor.h

Su función es separar la interfaz del procesamiento interno, lo cual ayuda a que el código sea más modular y fácil de mantener.

### data/sensor_data.csv

Este archivo contiene las muestras de prueba utilizadas por el programa. En este caso, se usaron cinco lecturas simuladas de un sistema IoT para validar el funcionamiento del análisis.

### reports/

La carpeta reports almacena el reporte generado por el programa. Dentro de ella se conserva un archivo .gitkeep para mantener la carpeta dentro del repositorio, aunque los reportes generados se excluyen mediante .gitignore.

## Explicación de Makefile

El Makefile inicia definiendo variables como CC, CFLAGS, TARGET, SRC_DIR, BUILD_DIR, DATA_DIR y REPORT_FILE. Estas variables permiten reutilizar rutas, comandos y nombres de archivos sin tener que escribirlos muchas veces. Esto vuelve el archivo más claro y fácil de modificar.

El objetivo **all** se encarga de compilar el programa principal. Al ejecutar make all, se generan los archivos objeto y posteriormente el ejecutable final.

El objetivo **help** muestra una lista de comandos disponibles. Esto facilita el uso del proyecto, ya que permite conocer rápidamente qué acciones se pueden ejecutar.

Las reglas **$(BUILD_DIR)** y **$(REPORT_DIR)** crean automáticamente las carpetas necesarias para almacenar los archivos compilados y el reporte generado. Esto evita que el usuario tenga que crear esos directorios manualmente.

**$(BUILD_DIR)/%.o: $(SRC_DIR)/%.c include/sensor.h | $(BUILD_DIR)**

> automatiza la compilación de cada archivo .c en su correspondiente archivo .o. Gracias a esto, no es necesario escribir una regla diferente para main.c y otra para sensor.c.

**$(TARGET): $(OBJS)**

> se encarga de enlazar los archivos objeto y generar el ejecutable final iot_report.exe.

**$(REPORT_FILE): $(TARGET) $(DATA_DIR)/sensor_data.csv | $(REPORT_DIR)**

> Esta indica que el reporte depende del ejecutable y del archivo CSV. Por lo tanto, si cambia el código fuente o cambian los datos de entrada, make puede volver a ejecutar el programa para actualizar el reporte.

El objetivo **run** depende del archivo de reporte, así que al ejecutarlo se compila el programa si es necesario, se ejecuta el análisis y se genera el reporte final.

El objetivo **report** muestra en consola el contenido del reporte generado, lo cual facilita revisar los resultados sin abrir manualmente el archivo.

El objetivo **clean** elimina los archivos generados, como los objetos .o, el ejecutable .exe y el reporte .txt. Esto permite limpiar el proyecto y reconstruirlo desde cero.

Finalmente, el objetivo **rebuild** combina **clean** y **all,** por lo que permite realizar una reconstrucción completa del proyecto.

## Comandos

```
C:\msys64\usr\bin\make.exe help
C:\msys64\usr\bin\make.exe all
C:\msys64\usr\bin\make.exe run
C:\msys64\usr\bin\make.exe report
C:\msys64\usr\bin\make.exe clean
C:\msys64\usr\bin\make.exe rebuild
```

Nota: En mi caso use la ruta completa de make porque así funcionó correctamente en MSYS2.

**Make help** se muestra la lista de objetivos disponibles

*C:\msys64\usr\bin\make.exe help*

```
Objetivos disponibles:
  make all      -> compila el ejecutable
  make run      -> compila, ejecuta y genera el reporte
  make report   -> muestra el reporte generado
  make clean    -> elimina archivos generados
  make rebuild  -> limpia y compila de nuevo
```

**Make all** se compiló correctamente el proyecto

*C:\msys64\usr\bin\make.exe all*

```
C:/msys64/ucrt64/bin/gcc.exe -Wall -Wextra -pedantic -std=c11 -O2 -Iinclude -c s
C:/msys64/ucrt64/bin/gcc.exe -Wall -Wextra -pedantic -std=c11 -O2 -Iinclude -c s
C:/msys64/ucrt64/bin/gcc.exe -Wall -Wextra -pedantic -std=c11 -O2 -Iinclude buil
```

**Make run** se genera automáticamente el reporte y se muestra el siguiente resumen

*C:\msys64\usr\bin\make.exe run*

```
./build/iot_report.exe data/sensor_data.csv reports/iot_report.txt
Analisis completado correctamente.
Muestras procesadas: 5
Temperatura promedio: 29.10 C
Humedad promedio: 40.60 %
CO2 promedio: 824.00 ppm
Reporte generado en: reports/iot_report.txt
```

**Make report** se obtiene el contenido del archivo generado

*C:\msys64\usr\bin\make.exe report*

```
Reporte de monitoreo IoT
========================
Muestras procesadas: 5
Temperatura promedio: 29.10 C
Humedad promedio: 40.60 %
CO2 promedio: 824.00 ppm
Temperatura maxima: 32.60 C
Alertas por temperatura (> 30.0 C): 2
Alertas por CO2 (> 900.0 ppm): 2
```

El uso de **make** permite trabajar con dependencias. Esto significa que si se modifica un archivo fuente, solo se recompila lo necesario. Si cambia el archivo CSV, el programa puede volver a generar el reporte actualizado.

En este proyecto, la automatización ayudó a reducir pasos manuales. En lugar de compilar cada archivo por separado, ejecutar el programa con sus argumentos y abrir manualmente el reporte, todo se puede realizar mediante comandos definidos en el Makefile.

Esto mejora el orden del proyecto, reduce errores y permite repetir el proceso de construcción de manera más confiable.

## Conclusión

Con esta práctica se desarrolló un proyecto propio en C utilizando un Makefile para automatizar el flujo de trabajo. El proyecto permitió compilar el código fuente, ejecutar el programa, procesar datos de un archivo CSV y generar un reporte final de manera automática.

Makefile no solo sirve para compilar programas, sino también para automatizar procesos completos dentro de un proyecto. En este caso, permitió integrar compilación, ejecución, generación de reportes, visualización de resultados y limpieza de archivos en un solo flujo ordenado y repetible.